

# Test SQL fenêtres

## 20 questions /20



ROW\_NUMBER, RANK, LAG/LEAD, partitions : ton vrai niveau intermédiaire. Durée conseillée : 20 min · 4 paliers · corrigé détaillé.

Objectif : vérifier si tu sais quand une fenêtre recommence à zéro, ce qu'un cadre par défaut change à un total cumulé, et pourquoi une fonction de fenêtre ne se filtre pas dans un WHERE.

**Repères :** 12/20 = bases posées, 17/20 = profil qu'on met sur un poste avec du reporting lourd.

Candidat \_\_\_\_\_ Date \_\_\_\_\_ Note \_\_\_\_\_/20

### Palier 1 : numéroter et classer.

*Cinq questions pour vérifier les bases du fenêtrage : ROW\_NUMBER, RANK, DENSE\_RANK, PARTITION BY, NTILE.*

**Question 1.** Deux commandes ont exactement le même montant et se retrouvent ex aequo au rang 2 après ORDER BY montant DESC. Quelle fonction renvoie 1, 2, 2, 4 (elle saute un rang après l'égalité) ?

- a) DENSE\_RANK()
- b) RANK()
- c) ROW\_NUMBER()
- d) NTILE(4)

**Question 2.** Tu veux classer les commandes de chaque client par date, en recommençant le classement à 1 pour chaque client. Quelle fenêtre écris-tu ?

- a) OVER (ORDER BY date)
- b) OVER (PARTITION BY date ORDER BY client\_id)
- c) GROUP BY client\_id ORDER BY date
- d) OVER (PARTITION BY client\_id ORDER BY date)

**Question 3.** SELECT client\_id, montant, ROW\_NUMBER() OVER (PARTITION BY client\_id ORDER BY montant DESC) AS rn FROM commandes. Que fait la colonne rn pour un client ayant 4 commandes ?

- a) Elle numérote de 1 à 4 les commandes de ce client, du montant le plus élevé au plus faible, en recommençant à 1 pour chaque nouveau client.
- b) Elle vaut 4 pour chaque ligne du client (nombre total de ses commandes).
- c) Elle numérote 1 à 4 seulement si aucun autre client n'existe dans la table.
- d) Elle renvoie une erreur car ROW\_NUMBER nécessite un GROUP BY.

**Question 4.** En entretien, on te demande si l'ORDER BY à l'intérieur d'un OVER() garantit aussi l'ordre d'affichage des lignes du résultat final.

- a) Oui, ORDER BY dans OVER() trie aussi le résultat final affiché.
- b) Non, il faut passer par GROUP BY pour trier.
- c) Non : ORDER BY dans OVER() ne trie que le calcul de la fenêtre, il faut un ORDER BY explicite hors fenêtre pour garantir l'ordre d'affichage.
- d) Oui, mais uniquement si la fenêtre contient un PARTITION BY.

**Question 5.** Tu veux répartir tes clients en 4 groupes de taille égale (quartiles) selon leur chiffre d'affaires. Quelle fonction choisis-tu ?

- a) RANK()
- b) NTILE(4)
- c) ROW\_NUMBER()
- d) DENSE\_RANK()

## Palier 2 : décalages LAG/LEAD.

*Cinq questions sur les décalages entre lignes : offset, valeur par défaut, LEAD, FIRST\_VALUE.*

**Question 6.** Dans `LAG(montant, 2) OVER (ORDER BY date)`, le 2 signifie :

- a) elle multiplie le montant par 2.
- b) elle regarde 2 lignes en avant.
- c) elle limite le résultat à 2 décimales.
- d) elle regarde 2 lignes en arrière.

**Question 7.** Pour la toute première commande de chaque client, `LAG(montant) OVER (PARTITION BY client_id ORDER BY date)` renvoie :

- a) NULL, sauf si une valeur par défaut est précisée en 3e argument de LAG.
- b) 0.
- c) une erreur d'exécution.
- d) la valeur de la commande elle-même.

**Question 8.** Pour comparer le montant d'une commande à celui de la commande suivante du même client, tu utilises :

- a) `LAG(montant) OVER (PARTITION BY client_id ORDER BY date)`
- b) `FIRST_VALUE(montant) OVER (PARTITION BY client_id)`
- c) `LEAD(montant) OVER (PARTITION BY client_id ORDER BY date)`
- d) `montant - montant`

**Question 9.** `SELECT client_id, date, FIRST_VALUE(montant) OVER (PARTITION BY client_id ORDER BY date) AS premier_montant FROM commandes.` Que renvoie `premier_montant` sur chaque ligne ?

- a) Le montant maximum du client.
- b) Le montant de la toute première commande (par date) de ce client, répété sur toutes ses lignes.
- c) Le montant de la ligne courante uniquement.
- d) Une erreur car `FIRST_VALUE` nécessite un `GROUP BY`.

**Question 10.** En entretien, on te demande le vrai avantage de LAG par rapport à une auto-jointure (self-join) pour comparer une ligne à la précédente.

- a) LAG est plus rapide dans tous les cas, sur tous les moteurs, sans exception.
- b) LAG fonctionne sans `ORDER BY`.
- c) LAG remplace complètement le besoin de `PARTITION BY`.
- d) LAG évite une jointure et le risque de doublons si la clé ou l'ordre n'est pas parfaitement unique.

## Palier 3 : le cadre qui piège.

*Chez Datafuji, la moitié des candidats qui écrivent une fonction de fenêtre syntaxiquement juste se plantent quand même sur le cadre par défaut en entretien technique. Six questions sur le cadre RANGE et ROWS, là où le chiffre devient faux sans erreur de syntaxe.*

**Question 11.** Dès qu'une fenêtre a un ORDER BY sans cadre explicite, le cadre par défaut appliqué par la plupart des moteurs est :

- a) RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
- b) ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
- c) ROWS BETWEEN CURRENT ROW AND UNBOUNDED FOLLOWING
- d) Aucun cadre n'est appliqué sans le préciser explicitement.

**Question 12.** Tu calcules un total cumulé avec `SUM(montant) OVER (ORDER BY date_commande)`. Deux commandes du même jour ont la même date\_commande. Le résultat :

- a) est correct : chaque ligne cumule bien les montants précédents un par un.
- b) ignore la deuxième commande du jour.
- c) additionne les deux commandes du même jour ensemble sur les deux lignes, à cause du cadre RANGE par défaut qui regroupe les valeurs égales de l'ORDER BY.
- d) renvoie NULL dès qu'une égalité apparaît dans l'ORDER BY.

**Question 13.** Pour obtenir un vrai cumul ligne par ligne, même avec des dates ex aequo, il faut écrire :

- a) Ajouter un GROUP BY date\_commande.
- b) `SUM(montant) OVER (ORDER BY date_commande ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW)`
- c) `SUM(montant) OVER (PARTITION BY date_commande)`
- d) Remplacer SUM par COUNT.

**Question 14.** `SELECT date, montant, LAST_VALUE(montant) OVER (ORDER BY date) AS dernier FROM commandes`. La colonne dernier ne renvoie pas le dernier montant de la table mais recopie souvent le montant de la ligne courante. La cause :

- a) LAST\_VALUE est buggé sur la plupart des moteurs.
- b) Il manque un PARTITION BY.
- c) LAST\_VALUE nécessite ORDER BY DESC pour fonctionner.
- d) Le cadre par défaut s'arrête à la ligne courante (RANGE ... AND CURRENT ROW); il faut forcer ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING.

**Question 15.** Quel est l'intérêt de `SUM(montant) OVER (PARTITION BY client_id)` par rapport à un GROUP BY client\_id classique ?

- a) Il garde une ligne par commande tout en affichant le total du client sur chaque ligne, sans replier le résultat.
- b) Aucun, les deux font strictement la même chose.
- c) Il est plus rapide dans tous les cas, sur tous les volumes.
- d) Il élimine automatiquement les doublons de commandes.

**Question 16.** Pour afficher, sur chaque ligne de commande, la part du montant de cette commande dans le total du client, tu écris :

- a) `montant / SUM(montant) GROUP BY client_id`
- b) `RANK() OVER (PARTITION BY client_id ORDER BY montant)`
- c) `montant / SUM(montant) OVER (PARTITION BY client_id)`
- d) `montant / COUNT(*) OVER (PARTITION BY client_id)`

## Palier 4 : filtrer une fenêtre.

Quatre questions sur ce qui bloque en pratique : filtrer une fonction de fenêtre, les NULL dans l'ORDER BY, la différence avec GROUP BY.

**Question 17. Un junior écrit** `SELECT * FROM commandes WHERE ROW_NUMBER() OVER (PARTITION BY client_id ORDER BY date DESC) = 1`. **La requête échoue. Pourquoi ?**

- a) ROW\_NUMBER n'existe pas en SQL standard.
- b) Il manque un GROUP BY avant le WHERE.
- c) PARTITION BY est interdit sans fonction d'agrégat classique.
- d) Les fonctions de fenêtre sont calculées après le WHERE; il faut une sous-requête, une CTE, ou QUALIFY selon le moteur.

**Question 18. Sur Snowflake ou BigQuery, la clause QUALIFY sert à :**

- a) Filtrer directement sur le résultat d'une fonction de fenêtre, sans passer par une sous-requête.
- b) Remplacer HAVING pour les agrégats classiques sans fenêtre.
- c) Trier les partitions par ordre alphabétique.
- d) Valider le type de données d'une colonne.

**Question 19. La colonne date\_commande contient quelques NULL. Dans** `ROW_NUMBER() OVER (ORDER BY date_commande)`, **ces lignes à NULL :**

- a) font échouer la requête avec une erreur de syntaxe.
- b) sont classées en général en premier ou en dernier selon le moteur : le comportement n'est pas universel, il faut le vérifier.
- c) sont automatiquement exclues de la fenêtre.
- d) reçoivent toutes le même rang 1.

**Question 20. La différence essentielle entre un agrégat classique (GROUP BY) et un agrégat fenêtré (OVER) est :**

- a) Le fenêtré est toujours plus lent que GROUP BY.
- b) OVER ne peut pas utiliser SUM ou COUNT.
- c) GROUP BY nécessite obligatoirement un ORDER BY.
- d) GROUP BY replie les lignes en une par groupe; OVER calcule l'agrégat mais garde toutes les lignes de détail.

## Solutions et explications.

*Un point par bonne réponse. Compte ton score, puis relis surtout celles que tu ne peux pas expliquer à voix haute, en particulier le piège du cadre RANGE par défaut.*

1. **b)** RANK laisse un trou après une égalité (1, 2, 2, 4); DENSE\_RANK ferait 1, 2, 2, 3 sans trou, et ROW\_NUMBER donnerait 1, 2, 3, 4 sans aucune égalité tolérée.
2. **d)** PARTITION BY redémarre le classement à 1 pour chaque client; sans lui, le classement courrait sur toute la table.
3. **a)** Le PARTITION BY isole chaque client, l'ORDER BY DESC classe ses commandes du montant le plus élevé au plus faible; rn redémarre à 1 pour chaque client.
4. **c)** ORDER BY dans OVER() ne pilote que le calcul interne de la fenêtre; sans ORDER BY externe au niveau de la requête, l'ordre d'affichage des lignes n'est pas garanti.
5. **b)** NTILE(4) répartit les lignes ordonnées en 4 groupes de taille aussi égale que possible; RANK ou ROW\_NUMBER ne répartissent pas en groupes.
6. **d)** Le deuxième argument de LAG précise le nombre de lignes à remonter dans la fenêtre ordonnée; LEAD ferait l'inverse et regarderait en avant.
7. **a)** Sans ligne précédente, LAG renvoie NULL par défaut; un 3e argument permet de forcer une valeur de repli comme 0.
8. **c)** LEAD regarde la ligne suivante dans l'ordre de la fenêtre; LAG regarderait la précédente, ce qui inverserait la comparaison.
9. **b)** FIRST\_VALUE avec un ORDER BY par date retourne la valeur de la première ligne de la partition et la répète sur toutes les lignes du client.
10. **d)** LAG lit la fenêtre déjà triée sans dupliquer de lignes; une auto-jointure sur une clé ou un ordre non strictement unique peut produire des doublons silencieux.
11. **a)** Sans cadre explicite mais avec un ORDER BY, la norme applique RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW, pas ROWS : c'est le piège des valeurs ex aequo.
12. **c)** Avec RANGE, toutes les lignes qui partagent la même valeur d'ORDER BY (ici la même date) sont traitées comme un seul palier et reçoivent le même cumul incluant les deux commandes du jour.
13. **b)** Forcer ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW rend le cumul strictement ligne par ligne, quelle que soit l'égalité de date\_commande.
14. **d)** LAST\_VALUE hérite du même cadre par défaut que les autres fonctions de fenêtre; sans un cadre étendu à UNBOUNDED FOLLOWING, elle ne voit jamais les lignes après la courante.
15. **a)** OVER conserve le grain de détail (une ligne par commande) tout en affichant un agrégat calculé sur la partition; GROUP BY replierait le résultat en une seule ligne par client.
16. **c)** Diviser le montant de la ligne par la somme fenêtrée du client donne la part de chaque commande dans le total, sans perdre le détail des lignes.
17. **d)** Les fonctions de fenêtre sont évaluées après le WHERE dans l'ordre logique d'exécution; il faut passer par une sous-requête, une CTE, ou QUALIFY quand le moteur le supporte.
18. **a)** QUALIFY est une clause propre à certains moteurs (Snowflake, BigQuery) qui filtre directement sur le résultat d'une fonction de fenêtre, en évitant l'enrobage en sous-requête.
19. **b)** Le classement des NULL dans un ORDER BY (première ou dernière position) dépend du moteur et n'est pas garanti par la norme SQL; il faut le vérifier ou le forcer explicitement.
20. **d)** GROUP BY replie chaque groupe en une seule ligne; OVER calcule le même agrégat mais garde toutes les lignes de détail, ce qui permet de comparer une ligne au total de son groupe.

## Fait main, en solo.

Si ce test t'a aidé, partage-le autour de toi : il peut débloquent un autre candidat. Et si tu veux que je traite un sujet précis, dis-le moi en commentaire : c'est là que je pioche mes prochains guides.

**Dylan**

